ANN

- The human brain contains hundreds of billions of cells called neurons. Each neuron has a cell body that processes information by receiving signals from other neurons (inputs) and sending signals to other neurons (outputs)
- Neural network consists of stack of neurons that takes in input x and output predicted value $h_0(x)$
- Matrix format: $z^l = W^l a^{l-1} + b^l$ and $a^l = g^l(z^l)$
- Logistic regression can be considered as a neural network without a hidden layer. Its mapping function connects the input x to the output y by a sigmoid function of a linear combination of features
- Terminology:
    - K: number of output neurons
    - L: total number of layers in network
    - s_l: number of neurons in layer l (not including bias)
    - For multiclass classification (K classes where k > 3):
        - $y \in R^k$
        - $h_\theta(x) \in R^k$
    - For binary classification:
        - $y \in \{0, 1\}$
        - $h_\theta(x) \in R$

Backpropagation

Forward pass:

$z^{(2)} = \theta^{(1)} a^{(1)}$

$a^{(2)} = g(z^{(2)})$

$z^{(3)} = \theta^{(2)} a^{(2)}$

$a^{(3)} = g(z^{(3)})$

$h_\theta(x) = a^{(3)}$

The Output layer error:

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}}$$

For a single training example,the mean squared error loss $J(\theta)$ is defined as:

$$J(\theta) = \frac{1}{2}(a^{(3)} - y)^2$$

where y is the target ground-truth label and $a^{(3)}$is the model's predicted activation output

$$\Rightarrow \frac{\delta J(\theta)}{\delta a^{(3)}} = \frac{1}{2} 2(a^{(3)} - y)^{2-1} \frac{d}{da^{(3)}} (a^{(3)} - y) = (a^{(3)} - y)$$

Since $a^{(3)} = g(z^{(3)})$

$$\Rightarrow \frac{\delta a^{(3)}}{\delta z^{(3)}} = \frac{g(z^{(3)})}{\delta z^{(3)}} = g'(z^{(3)})$$

Combining the terms:

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

The Output layer Weight Gradient:

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = \delta^{(3)} \frac{\delta z^{(3)}}{\delta \theta^{(2)}}$$

With $z^{(3)} = \theta^{(2)} a^{(2)} \Rightarrow \frac{\delta z^{(3)}}{\delta \theta^{(2)}} = a^{(2)}$

$$\Rightarrow \frac{\delta J(\theta)}{\delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

In vector-matrix form: $\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \delta^{(3)} (a^{(2)})^T$

(ensuring correct outer-product dimensions where $\delta^{(3)}$ is $m \times 1$ and $a^{(2)}$ is $n \times 1$)

The Hidden layer error:

$$\delta^{(2)} = \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta)}{\delta z^{(3)}} \frac{\delta z^{(3)}}{\delta a^{(2)}} \frac{\delta a^{(2)}}{\delta z^{(2)}}$$

Term 1 is the error of the next layer: $\frac{\delta J(\theta)}{\delta z^{(3)}} = \delta^{(3)}$

Term 2 becomes $\frac{\delta z^{(3)}}{\delta a^{(2)}} = \theta^{(2)}$ because $z^{(3)} = \theta^{(2)} a^{(2)}$

Term 3 becomes $\frac{\delta a^{(2)}}{\delta z^{(2)}} = g'(z^{(2)})$ because $a^{(2)} = g(z^{(2)})$

$\Rightarrow \delta^{(3)}\theta^{(2)}g'(z^{(2)}) \Leftrightarrow \delta^{(3)}(\theta^{(2)})^{T}g'(z^{(2)})$

(ensuring $\delta^{(3)}$ is $m \times 1$ and $\theta^{(2)}$ is $m \times n$ and $g'(z^{(2)})$ is $n \times 1$)

- $\delta^{(3)}(\theta^{(2)})^{T}$ projects the output error $\delta^{(3)}$ backward through the weights of layer 2 to see how much each hidden neuron contributed to that error
- $g'(z^{(2)})$ scales that projected error by how sensitive the hidden neuron's activation function g was to its input at that operating point

---

Input layer Weight Gradient:

$$\frac{\delta J(\theta)}{\delta \theta^{(1)}} = \frac{\delta J(\theta)}{\delta z^{(3)}}\frac{\delta z^{(3)}}{\delta a^{(2)}}\frac{\delta a^{(2)}}{\delta z^{(2)}}\frac{\delta z^{(2)}}{\delta \theta^{(1)}}$$

Term 1 + 2 + 3 is the $\delta^{(2)}$

Term 4 becomes $\frac{\delta z^{(2)}}{\delta \theta^{(1)}} = a^{(1)}$ because $z^{(2)} = \theta^{(1)}a^{(1)}$

$$\Rightarrow \frac{\delta J(\theta)}{\delta \theta^{(1)}} = \delta^{(2)}a^{(1)} \Leftrightarrow \delta^{(2)}(a^{(1)})^{T}$$

(ensuring $\delta^{(2)}$ is $m \times 1$ and $a^{(1)}$ is $n \times 1$)

Notice the structural pattern across all weight gradients in backpropagation:

$$\frac{\delta J(\theta)}{\delta \theta^{(l)}} = \delta^{(l+1)}(a^{(l)})^{T}$$

⇒ *The gradient for any weight matrix* $\theta^{(l)}$ *is always the error term of the receiving layer* $\delta^{(l+1)}$ *multiplied by the transposed activations of the sending layer* $(a^{(l)})^{T}$

Why NO $\delta^{(1)}$?

$\delta^{(l)}$ represents the error associated with layer l's weighted inputs ($z^{(l)}$). Because layer 1 is the input layer containing raw input features rather than computed activations from weights, there is no $z^{(1)}$ to differentiate against, and input values cannot be adjusted via gradient descent

Backpropagation algorithm:

1. **Input:** Given N number of data points (training set):
   $(x^{(1)}, y^{(1)}),\ (x^{(2)}, y^{(2)}),\ \dots\ , (x^{(n)}, y^{(n)})$
2. **Feedforward:** For each $l = 1, 2, \dots, L$, compute $z^{(l+1)} = \theta^{(l)} a^{(l)}$ and $a^{(l+1)} = g(z^{(l+1)})$
3. **Output error:** Compute $\delta^{(L)}$
4. **Backpropagate:** For each $l = 2, \dots, L$, compute $\delta^{(l)}$
5. **Gradient computation:** Compute the gradients for the weights and biases
6. **Optimization algorithm:** Can use any gradient descent optimization algorithm to minimize the error function $J(\theta)$

Random Initialization of Weights:

- At the start of training, neural network weights are initialized with random values
- Random initialization helps neurons learn different features

LR vs ANN

- Neural Network:
  - Neural networks are much better for a **complex nonlinear hypothesis**
  - A neural network is in general **not convex**. It can suffer from multiple local minima
- Logistic Regression:
  - LR is useful for dataset where the classes are “linearly separable”
  - For relatively very **small dataset**
  - It is a great, robust model for **simple classification tasks**
  - LR cost function is **convex**, and thus we are guaranteed to find the global cost minimum